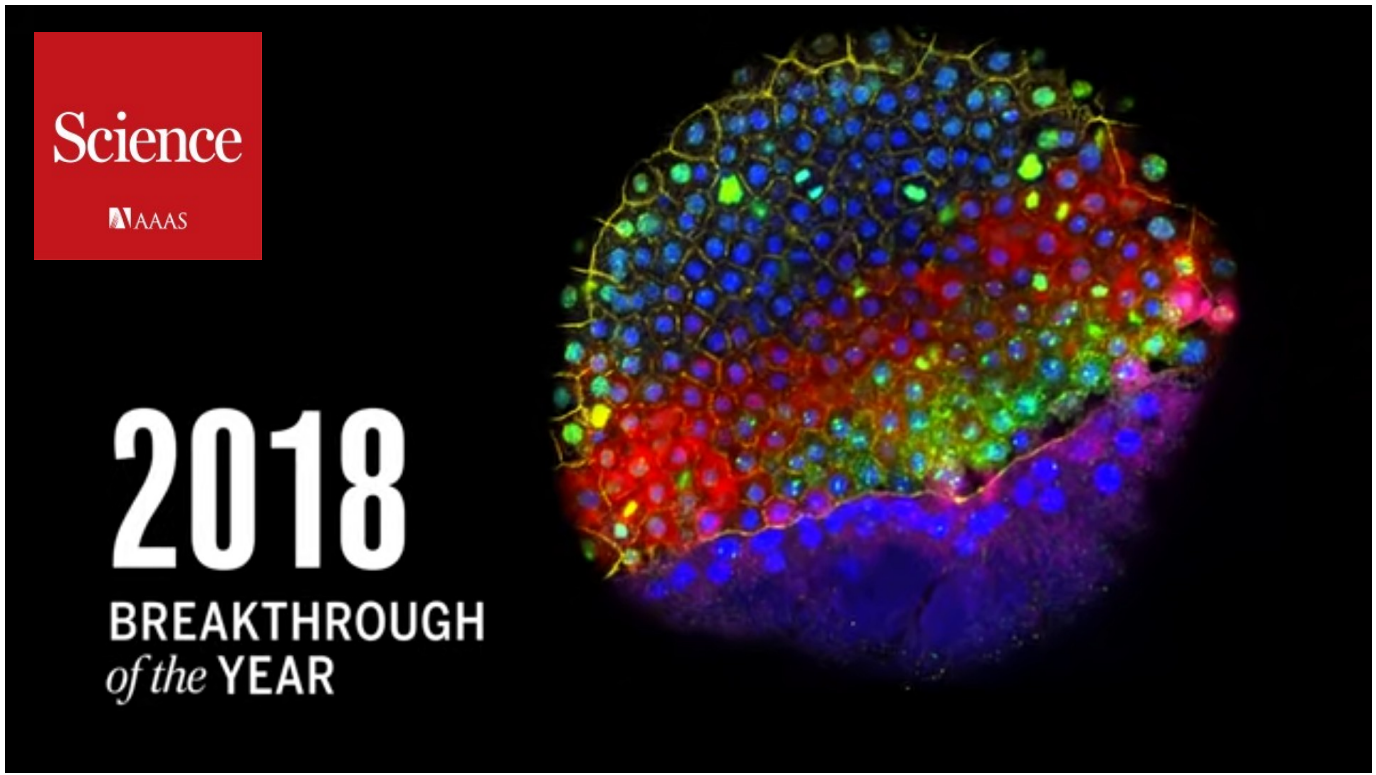
Single Cell RNAseq Analysis

Nikolay Oskolkov, Metabolic Research Group leader at NIRI, Riga, Latvia

PR Stats course, **Session 7: clustering scRNAseq data**



@NikolayOskolkov

@oskolkov.bsky.social

Personal homepage:
https://nikolay-oskolkov.com

Nature method of the year:
2019 (single cell multi-omics),
2020 (spatial transcriptomics)

Image adapted from Science, 362, 6421, 2018

Session content

Topics we'll cover in this session:

1) Basic clustering: hierarchical + partitioning (k-means, GMM)

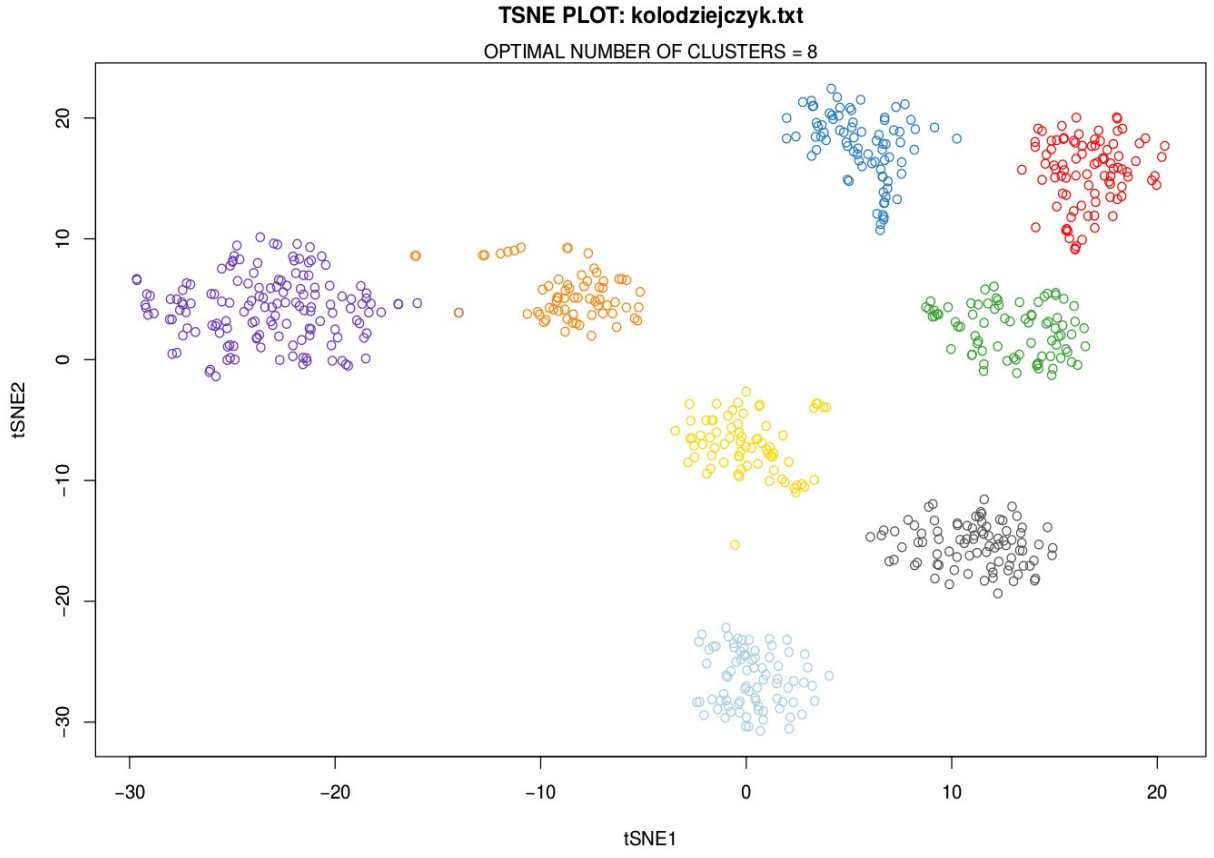
2) Advanced clustering: spectral and density-based clustering

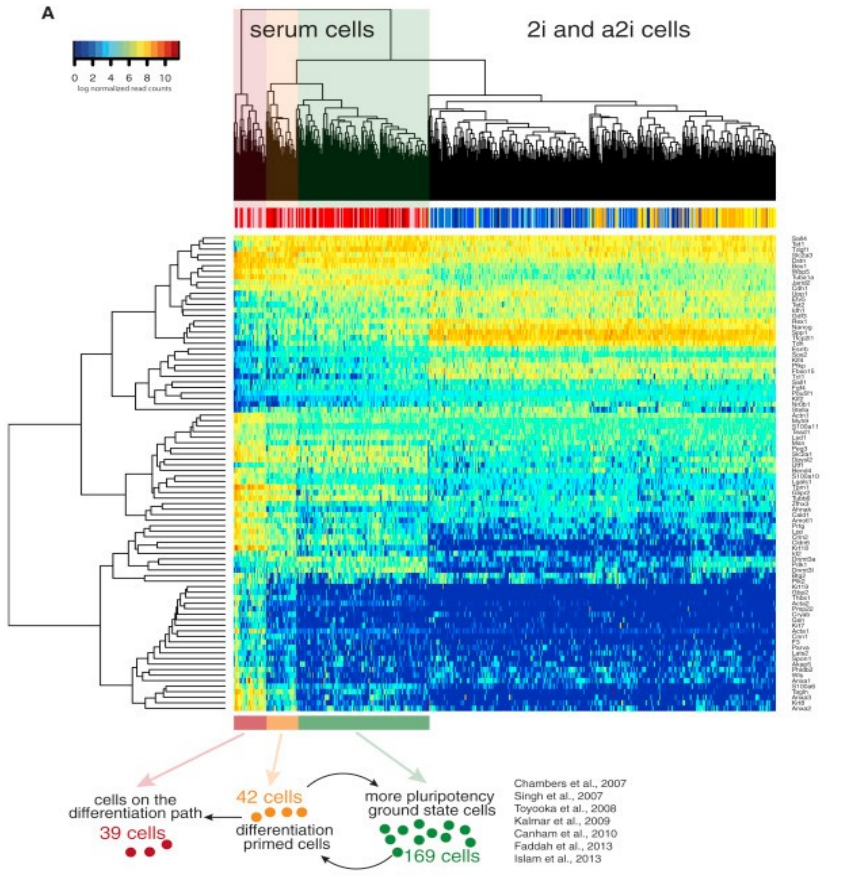
3) Graph-based clustering and comparing clustering methods

Cluster Analysis

1) Hierarchical clustering
2) Centroid - based clustering
3) Graph - based clustering
4) Density-based clustering

When clustering disagrees with dim reduction

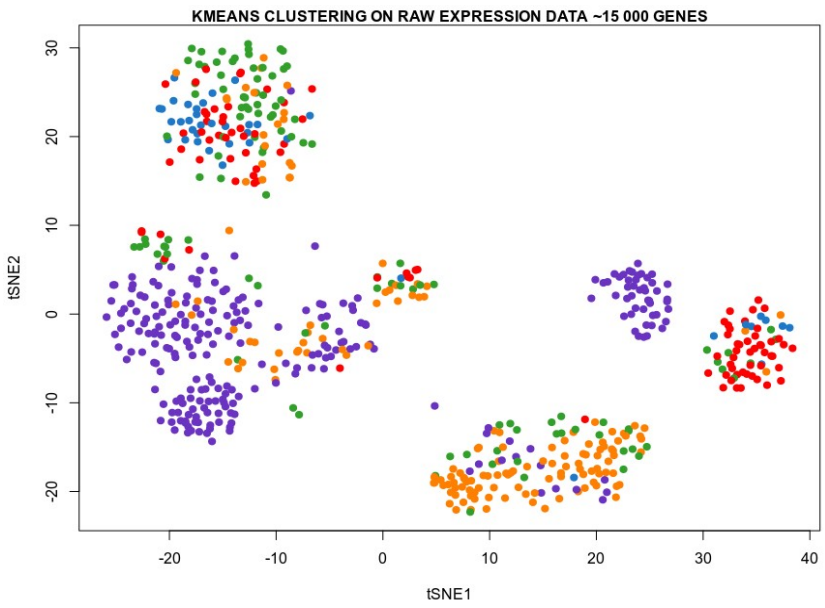




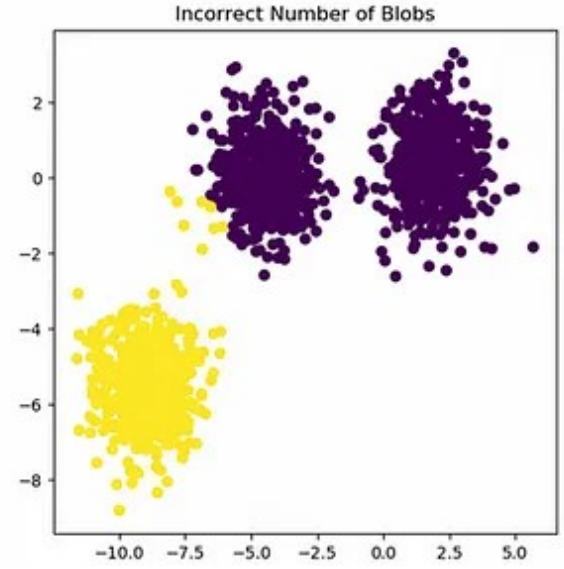
Kolodziejczyk et al., Cell Stem Cell. 2015

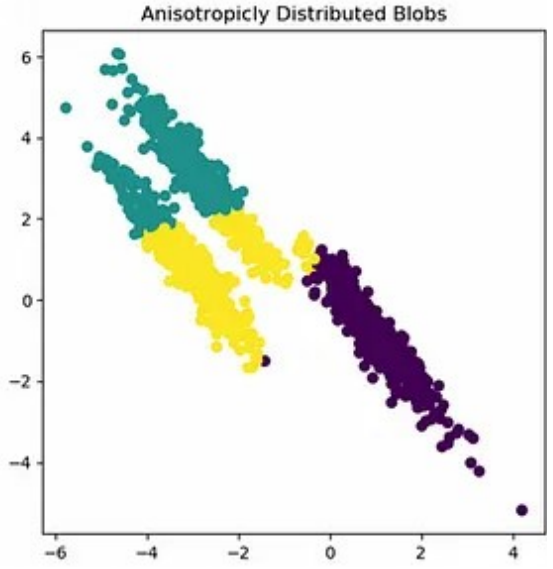
Why do I see 8 clusters, while 3 cell types are reported in the publication?

More serious failures of cluster analysis



Cluster analysis assigns data points to clusters
that cannot be visually validated by dimension reduction



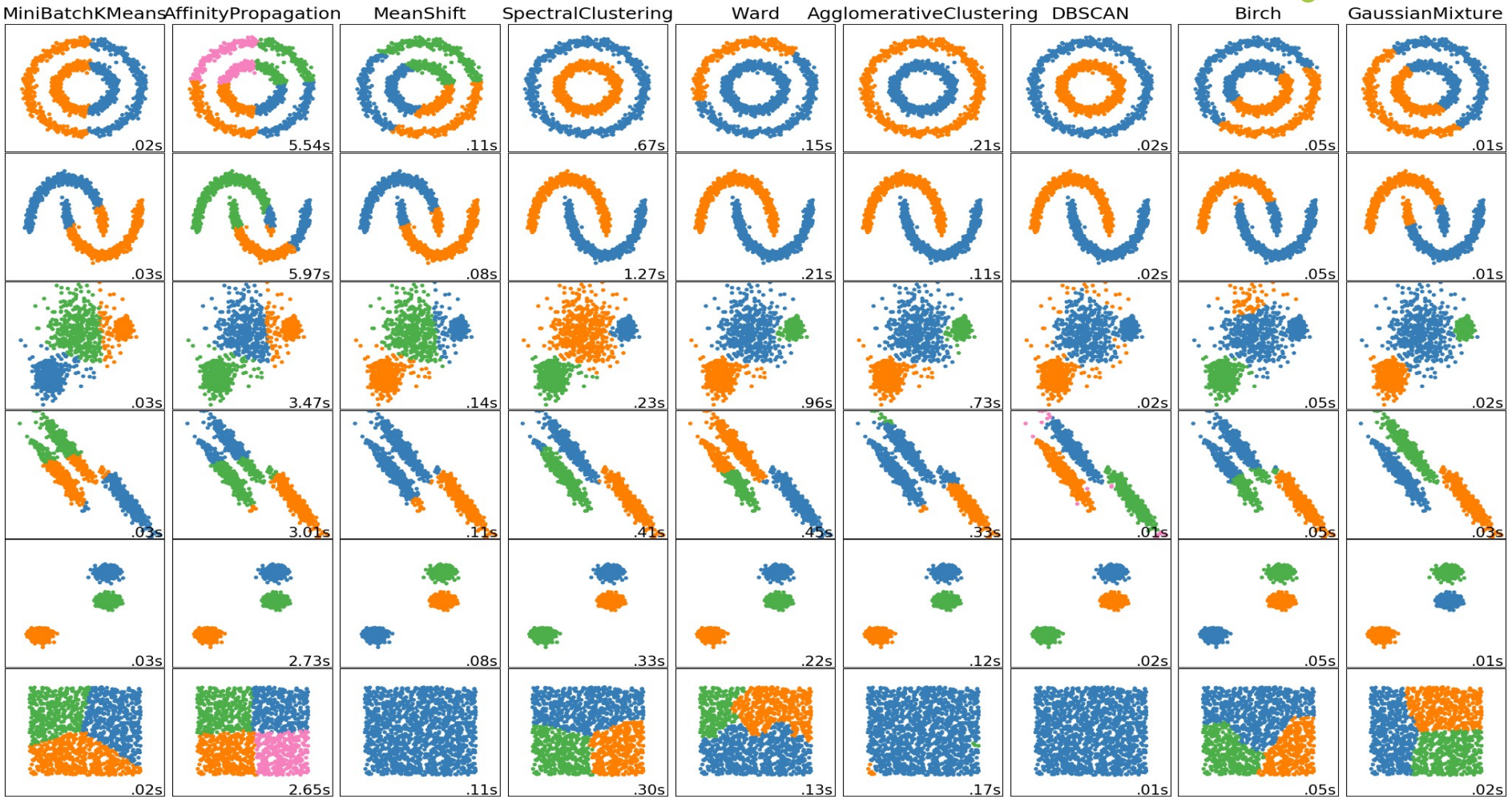


Cluster analysis collapses or splits clusters
which are visually apparent from simple 2D data plots

Which method is correct: dimension reduction (i.e. what I see) or clustering?

The Curse of Dimensionality is a potential explanation for such clustering failures

Huge variety of clustering methods available

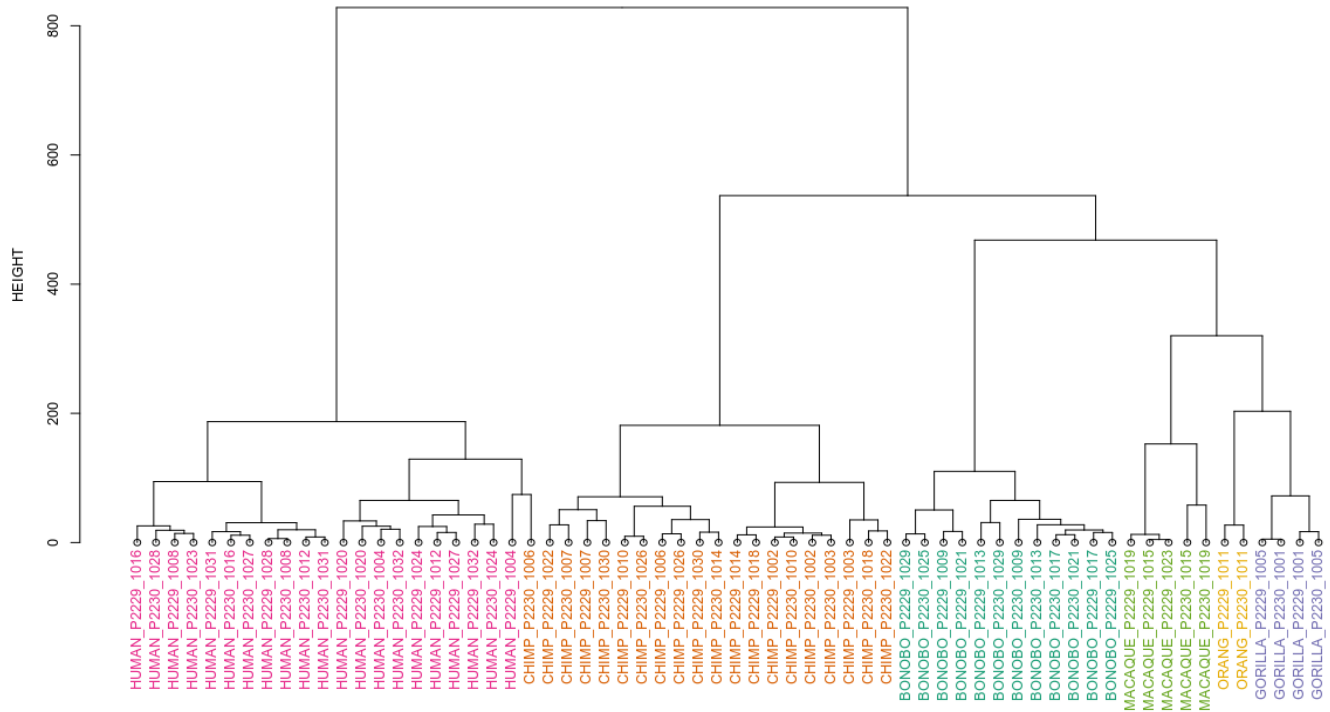


1) Hierarchical Clustering, 2) Partitioning Clustering, 3) Density-Based Clustering, 4) Graph-Based Clustering

Hierarchical clustering

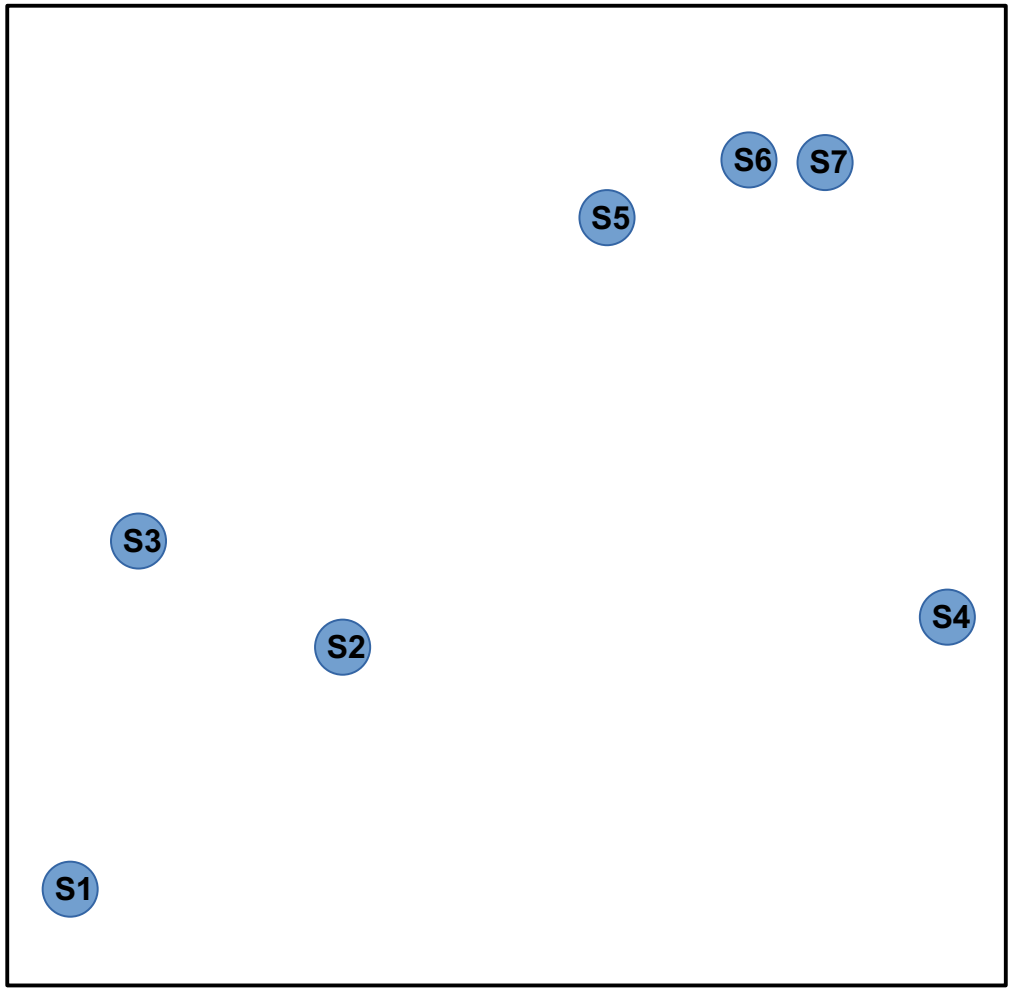
What is hierarchical clustering?

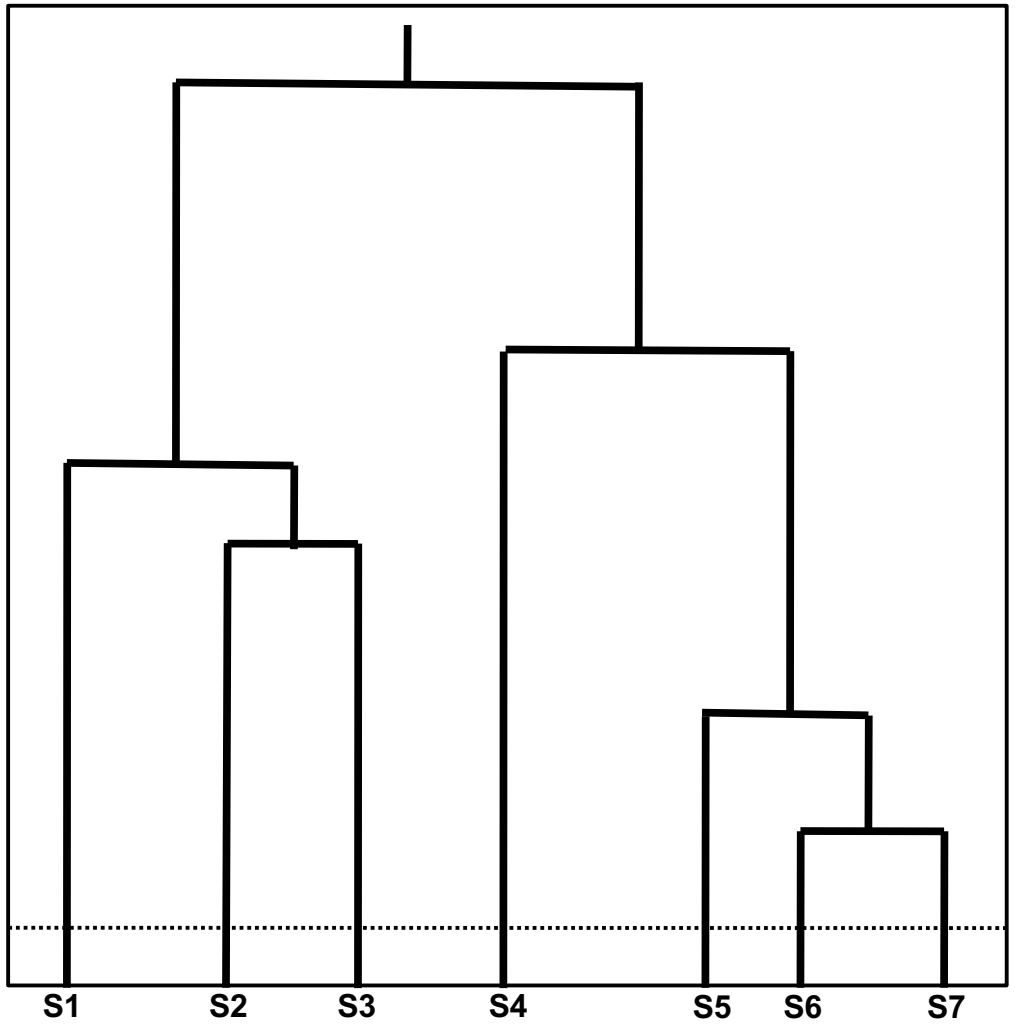
Clusters are nested (higher-ranked clusters contain lower-ranked clusters),
in other words tree-like structure (dendrogram)



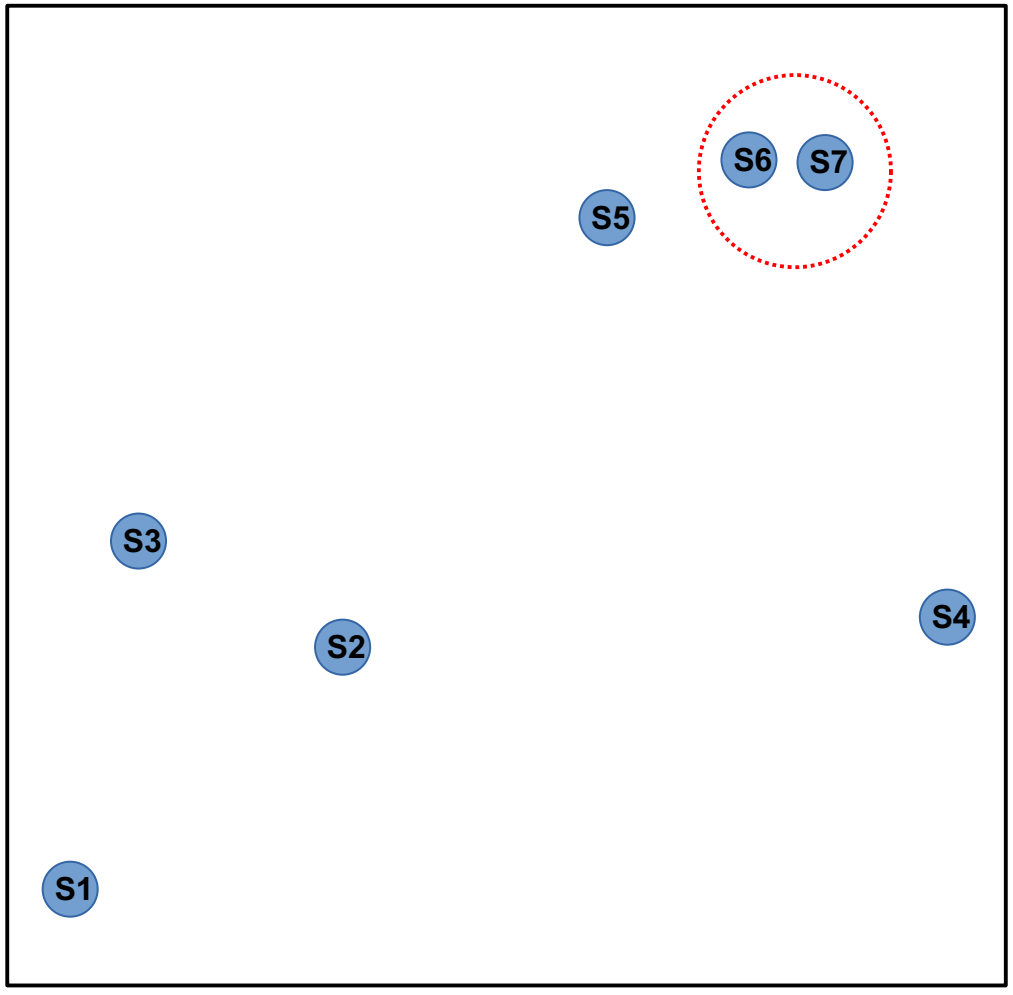
There are agglomerative (bottom-up approach) and divisive (top-down approach) hierachical types of hierachical clustering

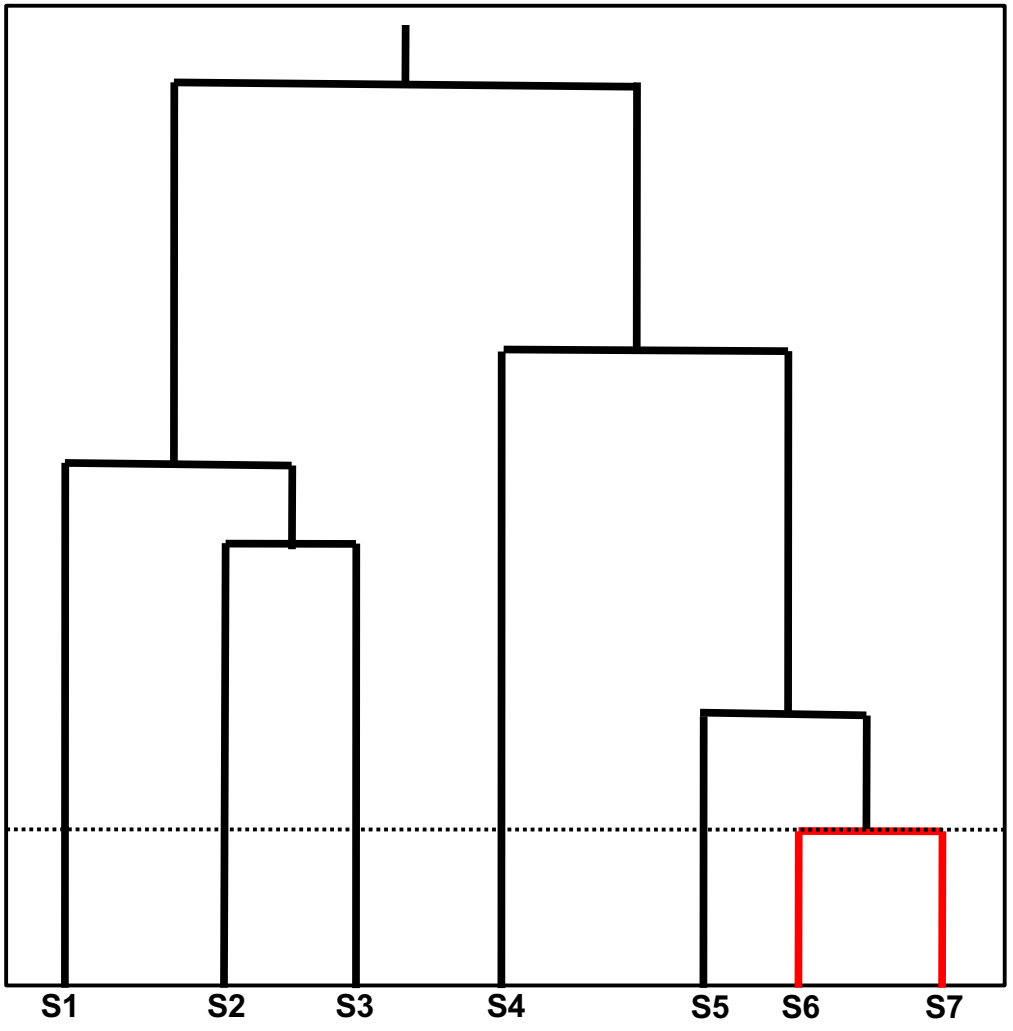
Agglomerative hierarchical clustering



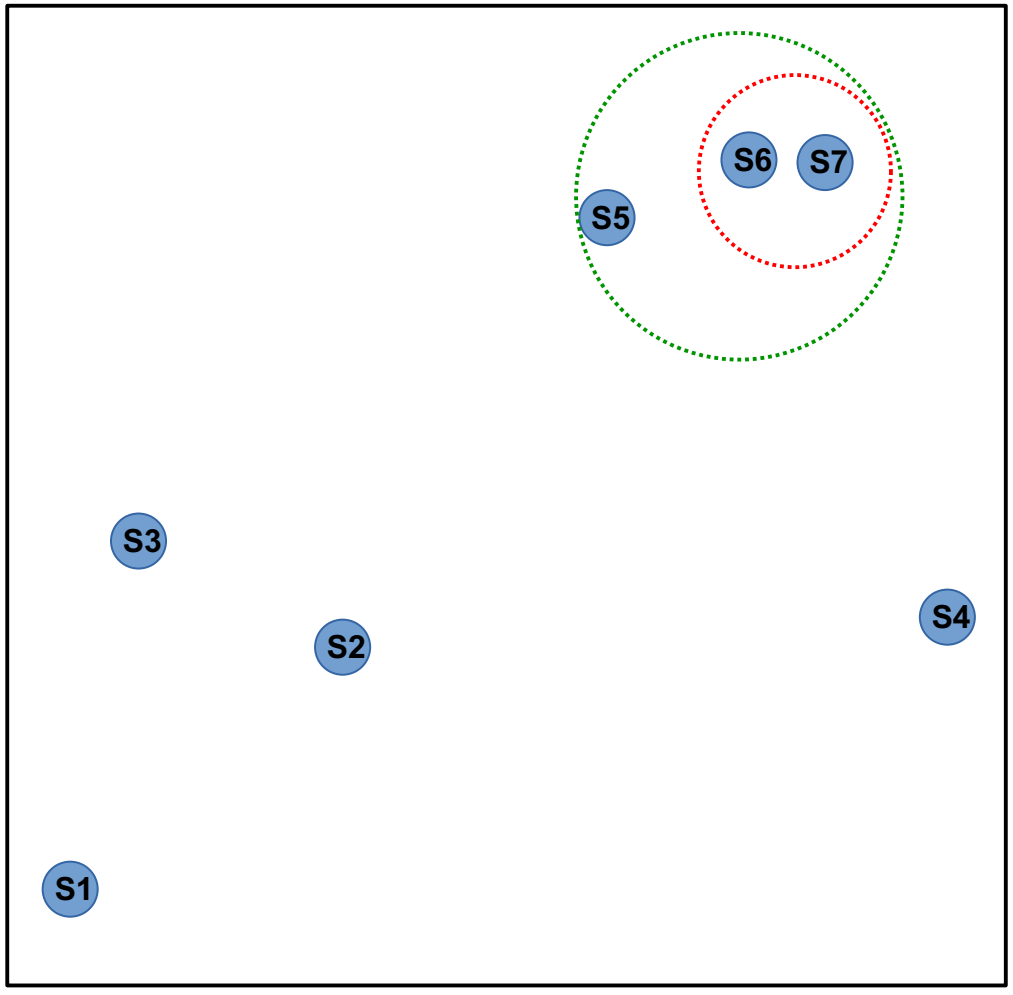


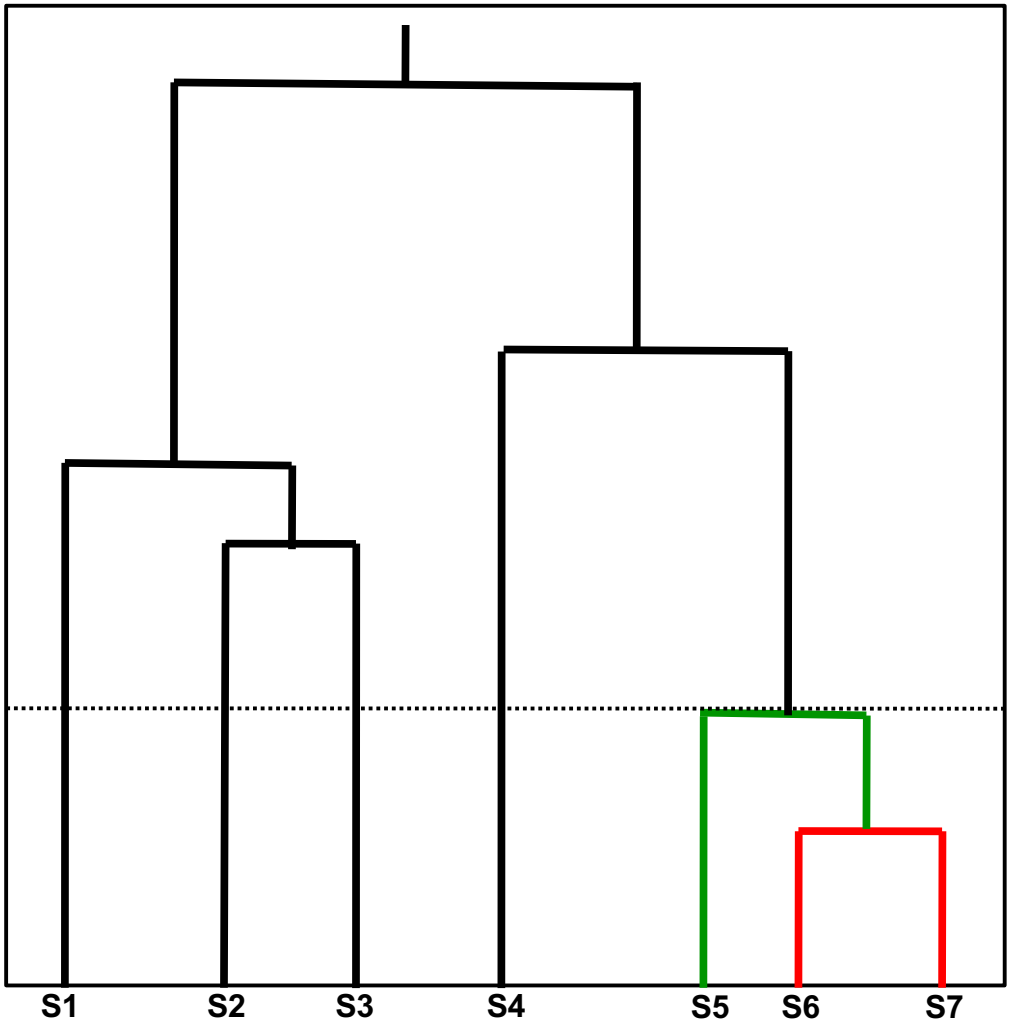
Agglomerative hierarchical clustering



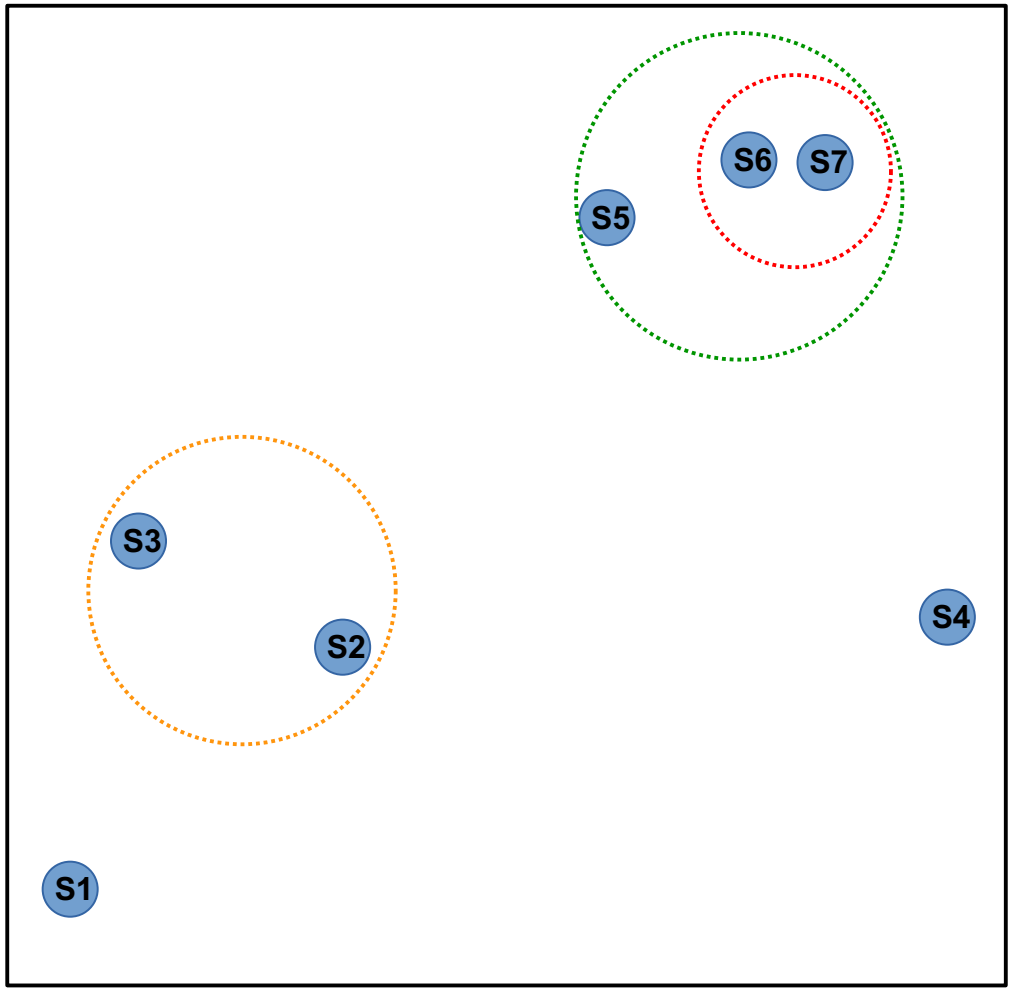


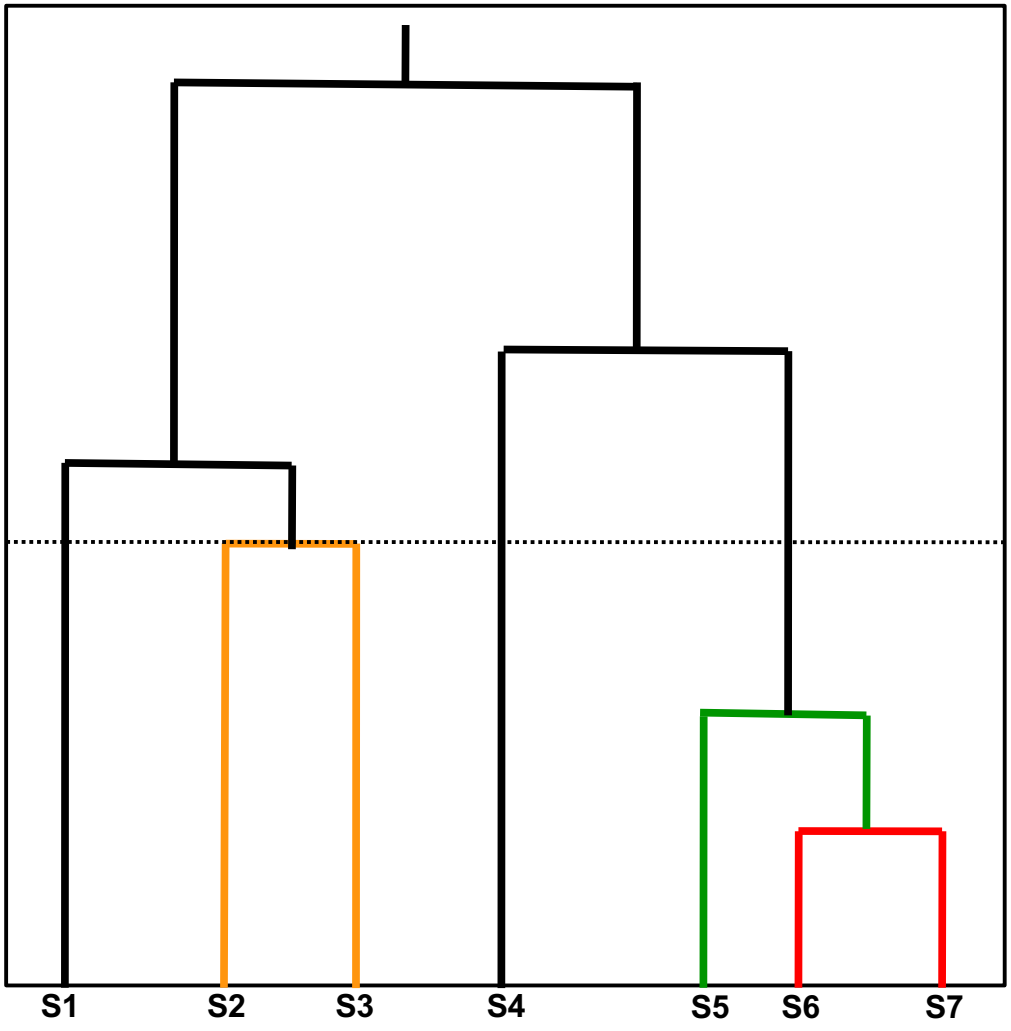
Agglomerative hierarchical clustering



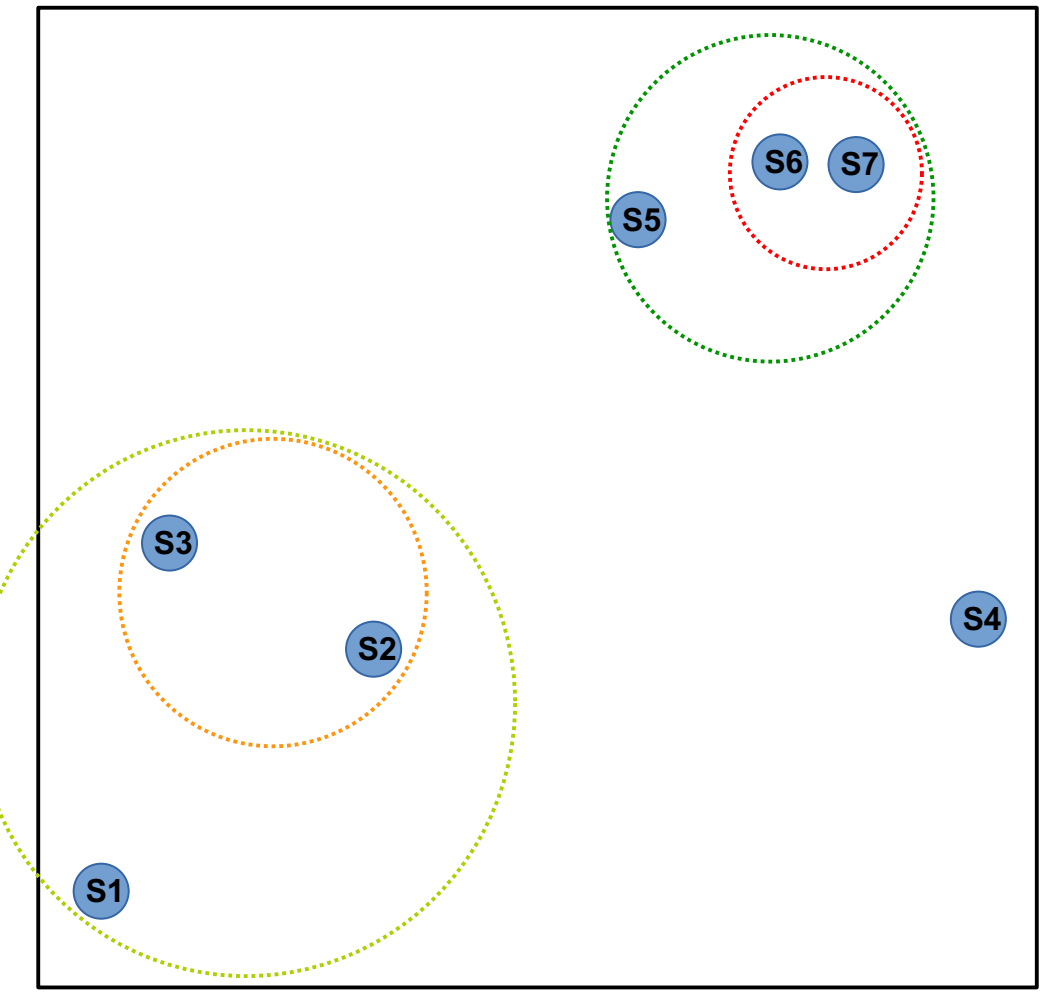


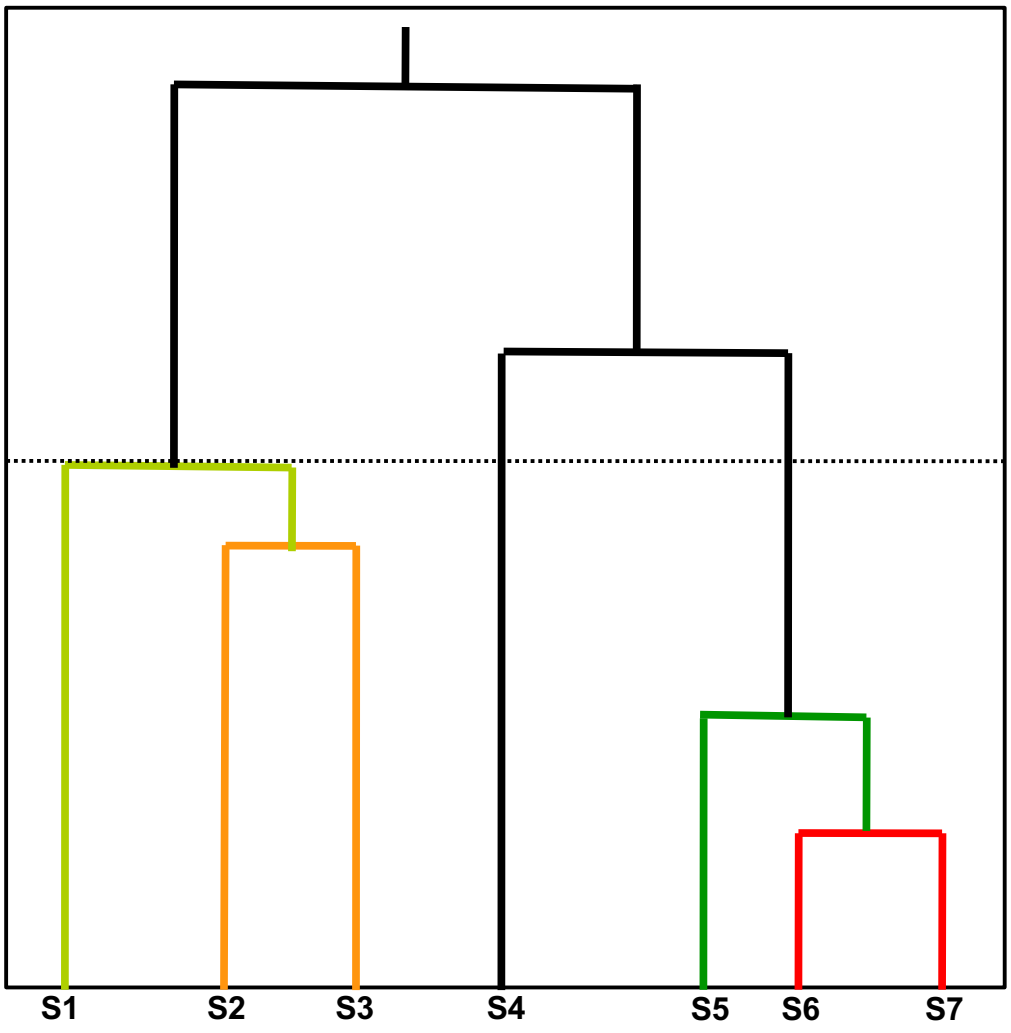
Agglomerative hierarchical clustering





Agglomerative hierarchical clustering





Activities RStudio sön 15:05 sv

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins Project: (None)

hclust_toy.R

Source on Save Run Source

```
1  df <- data.frame(x = c(0, 5, 2, 12, 6, 8, 7), y = c(0, 3, 5, 3, 8, 9, 9))
2  df
3  plot(y ~ x, data = df, pch = 16, cex = 5, col = 2:8, xaxt="n", yaxt="n")
4  axis(1, at = seq(0, 12, by = 1)); axis(2, at = seq(0, 9, by = 1))
5  abline(h = 0:9, v = 0:12, lty = 1, col = "lightgrey")
6  points(y ~ x, data = df, pch = 16, cex = 5, col = 2:8)
7  points(y ~ x, data = df, pch = as.character(1:7))
8
9  dist(df)
10 clusters <- hclust(dist(df))
11 plot(as.dendrogram(clusters), ylab = "Euclidean distance")
12
```

12:1 (Top Level) R Script

Console Terminal Background Jobs

R 4.2.3 · ~/

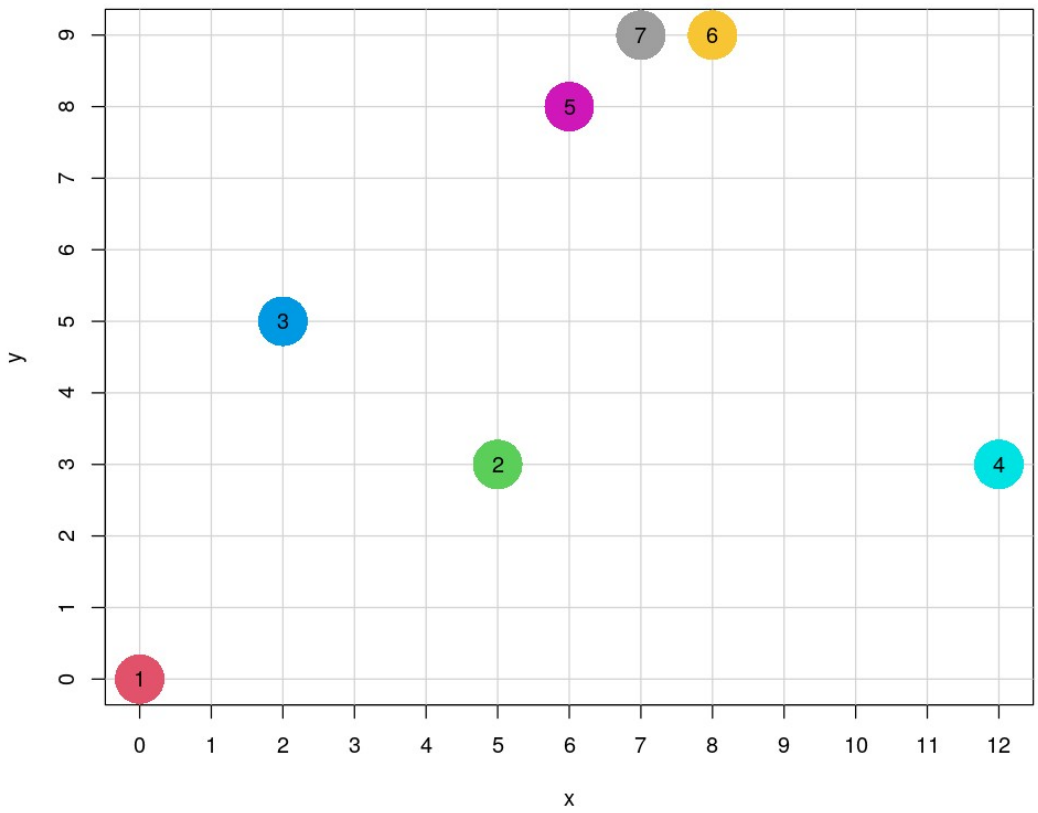
```
> df <- data.frame(x = c(0, 5, 2, 12, 6, 8, 7), y = c(0, 3, 5, 3, 8, 9, 9))
> df
   x y
1  0 0
2  5 3
3  2 5
4 12 3
5  6 8
6  8 9
7  7 9
> plot(y ~ x, data = df, pch = 16, cex = 5, col = 2:8, xaxt="n", yaxt="n")
> axis(1, at = seq(0, 12, by = 1)); axis(2, at = seq(0, 9, by = 1))
> abline(h = 0:9, v = 0:12, lty = 1, col = "lightgrey")
> points(y ~ x, data = df, pch = 16, cex = 5, col = 2:8)
> points(y ~ x, data = df, pch = as.character(1:7))
> dist(df)
         1        2        3        4        5        6
2  5.830952
3  5.385165  3.605551
4 12.369317  7.000000 10.198039
5 10.000000  5.099020  5.000000  7.810250
6 12.041595  6.708204  7.211103  7.211103  2.236068
7 11.401754  6.324555  6.403124  7.810250  1.414214  1.000000
> clusters <- hclust(dist(df))
> plot(as.dendrogram(clusters), ylab = "Euclidean distance")
>
```

Environment History Files Packages

Plots Connections Help Viewer Presentation

Zoom Export Publish

euc_dist_23 = sqrt ((2-5)^2 + (5-3)^2) = 3.605551



Activities RStudio sön 15:06 sv

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins Project: (None)

hclust_toy.R

Source on Save Run Source

```
1  df <- data.frame(x = c(0, 5, 2, 12, 6, 8, 7), y = c(0, 3, 5, 3, 8, 9, 9))
2  df
3  plot(y ~ x, data = df, pch = 16, cex = 5, col = 2:8, xaxt="n", yaxt="n")
4  axis(1, at = seq(0, 12, by = 1)); axis(2, at = seq(0, 9, by = 1))
5  abline(h = 0:9, v = 0:12, lty = 1, col = "lightgrey")
6  points(y ~ x, data = df, pch = 16, cex = 5, col = 2:8)
7  points(y ~ x, data = df, pch = as.character(1:7))
8
9  dist(df)
10 clusters <- hclust(dist(df))
11 plot(as.dendrogram(clusters), ylab = "Euclidean distance")
12
```

12:1 (Top Level) R Script

Console Terminal Background Jobs

R 4.2.3 · ~/

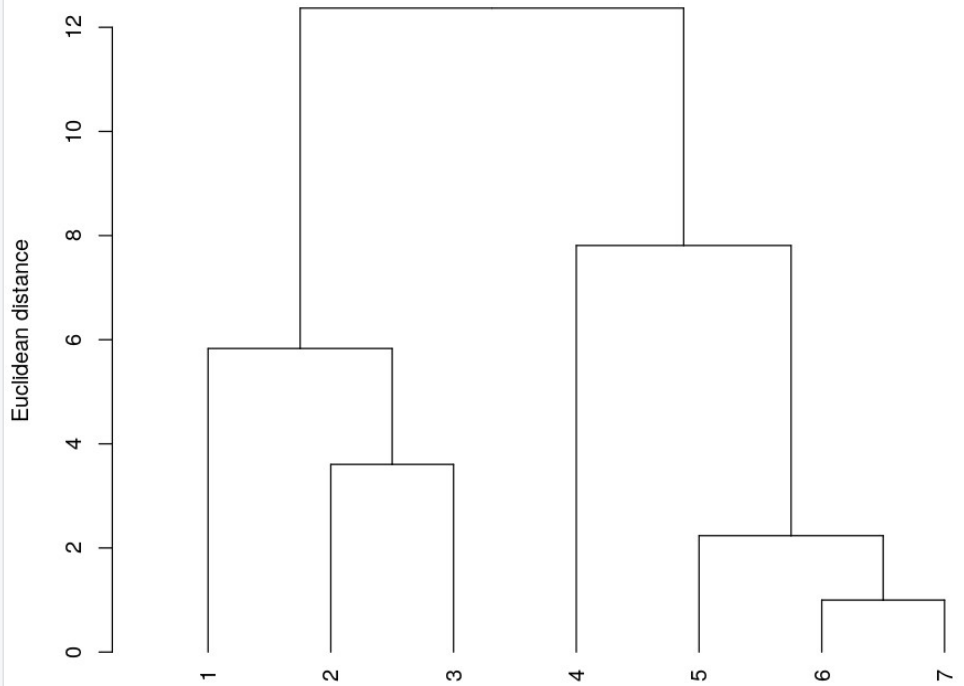
```
> df <- data.frame(x = c(0, 5, 2, 12, 6, 8, 7), y = c(0, 3, 5, 3, 8, 9, 9))
> df
   x y
1  0 0
2  5 3
3  2 5
4 12 3
5  6 8
6  8 9
7  7 9
> plot(y ~ x, data = df, pch = 16, cex = 5, col = 2:8, xaxt="n", yaxt="n")
> axis(1, at = seq(0, 12, by = 1)); axis(2, at = seq(0, 9, by = 1))
> abline(h = 0:9, v = 0:12, lty = 1, col = "lightgrey")
> points(y ~ x, data = df, pch = 16, cex = 5, col = 2:8)
> points(y ~ x, data = df, pch = as.character(1:7))
> dist(df)
         1        2        3        4        5        6
2  5.830952
3  5.385165  3.605551
4 12.369317  7.000000 10.198039
5 10.000000  5.099020  5.000000  7.810250
6 12.041595  6.708204  7.211103  7.211103  2.236068
7 11.401754  6.324555  6.403124  7.810250  1.414214  1.000000
> clusters <- hclust(dist(df))
> plot(as.dendrogram(clusters), ylab = "Euclidean distance")
>
```

Environment History Files Packages

Plots Connections Help Viewer Presentation

Zoom Export Publish

euc_dist_23 = sqrt ((2-5)^2 + (5-3)^2) = 3.605551



Types of hierarchical clustering

Add sample to cluster by:

1) Closest neighbor in cluster (single linkage)

2) Farthest neighbor in cluster (complete linkage)

3) Median / mean of cluster (UPGMA / UPGMC)

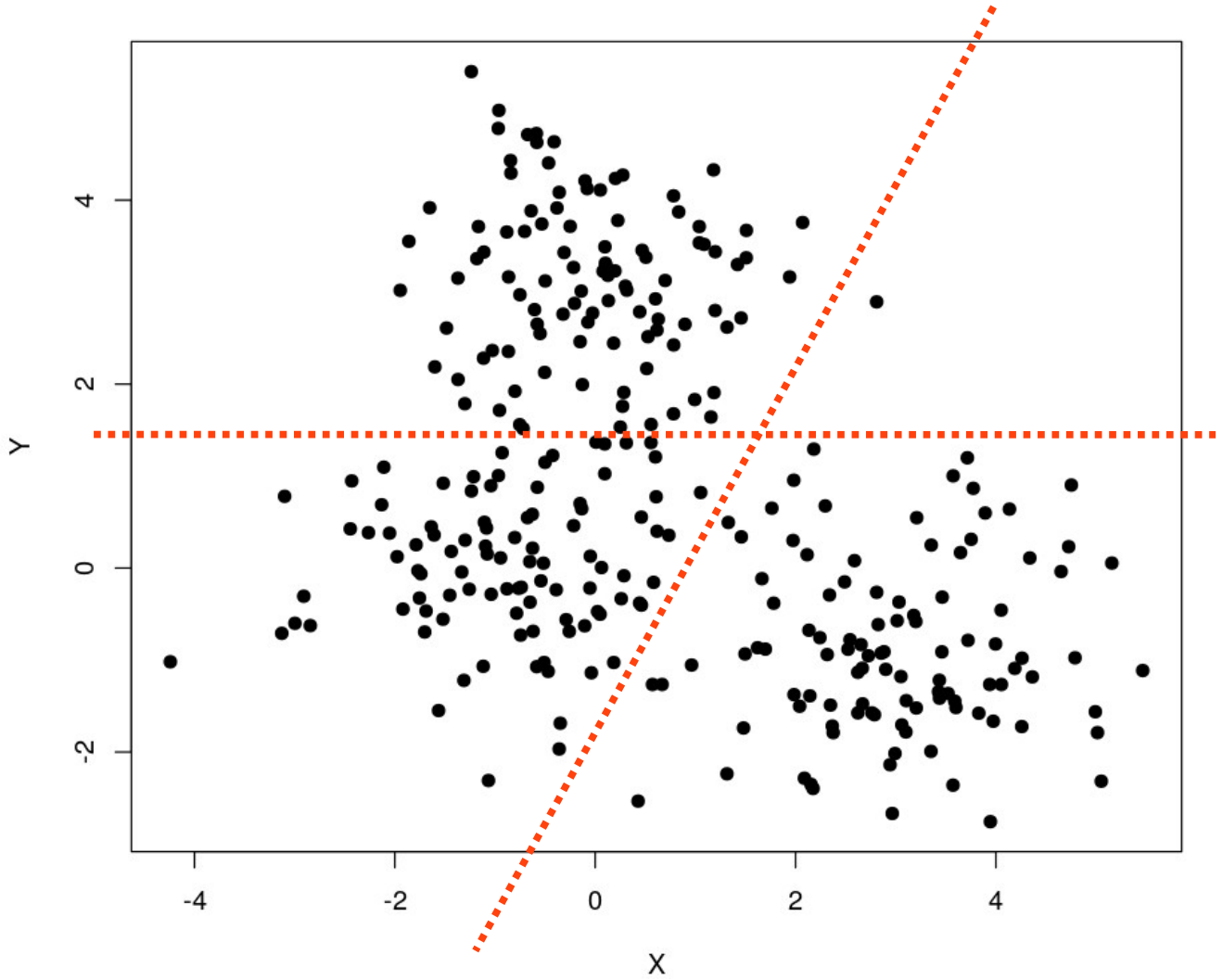
4) Least change in variation (Ward)

Disadvantages of hierachical clustering (no matter type):

Very sensitive to noise in data

Partitioning clustering

Partitioning principle



Assign points to K = 2, 3, 4, 5, … groups (clusters)

No dendrogram!

K-means, C-means, PAM, GMM, ...

Activities RStudio sön 15:35 sv

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins Project: (None)

Kmeans_from_scratch.R

Source on Save Run Source

```
1  library("MASS")
2
3  set.seed(123)
4
5  data1 <- as.data.frame(mvrnorm(n = 100, mu = c(-1, 0), Sigma=matrix(c(1, 0, 0, 1), ncol = 2)))
6  data2 <- as.data.frame(mvrnorm(n = 100, mu = c(3, -1), Sigma=matrix(c(1, 0, 0, 1), ncol = 2)))
7  data3 <- as.data.frame(mvrnorm(n = 100, mu = c(0, 3), Sigma=matrix(c(1, 0, 0, 1), ncol = 2)))
8  data<-rbind(data1, data2, data3)
9  head(data)
10 plot(data, xlab = "X", ylab = "Y", pch = 19)
11
12 par(mfrow=c(2, 2))
13 K <- 3
14 set.seed(123)
15 centers <- data[sample(1:dim(data)[1], K), ]
16 plot(data, xlab = "X", ylab = "Y", pch = 19)
17 points(centers, col="blue", cex = 3, pch = 19)
18 for(t in 1:3)
```

12:1 (Top Level) R Script

Console Terminal Background Jobs

R 4.2.3 · ~/

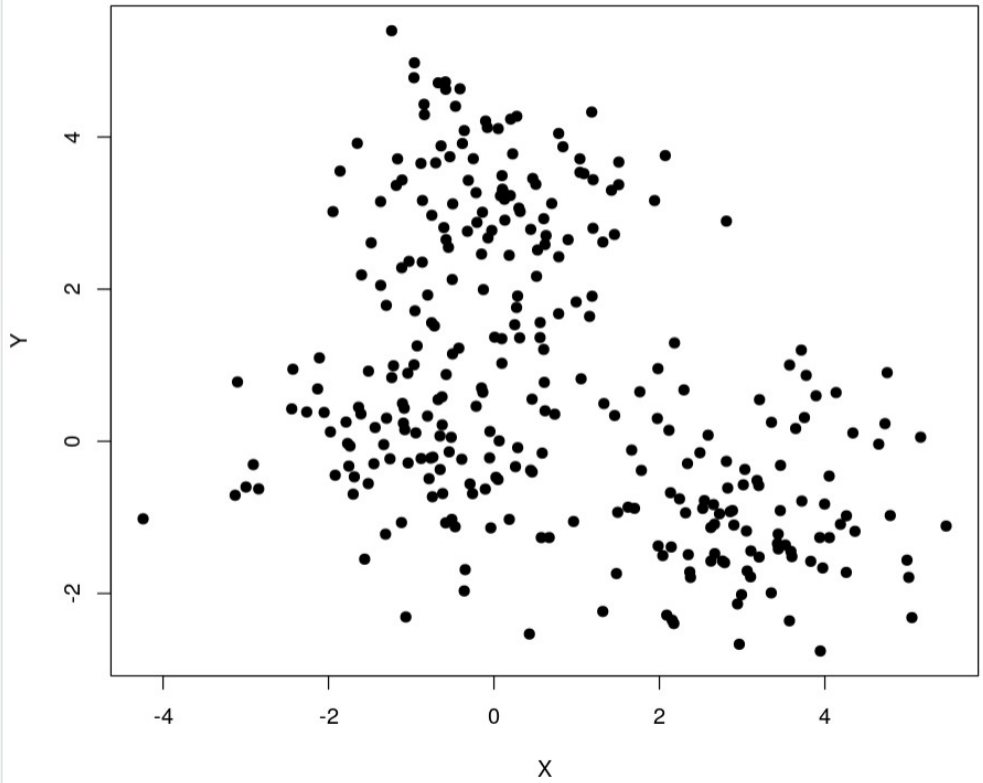
```
>
>
>
> library("MASS")
> set.seed(123)
> data1 <- as.data.frame(mvrnorm(n = 100, mu = c(-1, 0), Sigma=matrix(c(1, 0, 0, 1), ncol = 2)))
> data2 <- as.data.frame(mvrnorm(n = 100, mu = c(3, -1), Sigma=matrix(c(1, 0, 0, 1), ncol = 2)))
> data3 <- as.data.frame(mvrnorm(n = 100, mu = c(0, 3), Sigma=matrix(c(1, 0, 0, 1), ncol = 2)))
> data<-rbind(data1, data2, data3)
> head(data)
          V1          V2
1 -0.28959344 -0.56047565
2 -1.25688371 -0.23017749
3 -0.75330812  1.55870831
4 -0.65245740  0.07050839
5 -0.04838143  0.12928774
6 -0.95497228  1.71506499
> plot(data, xlab = "X", ylab = "Y", pch = 19)
>
```

Environment History Files Packages

R Global Environment

Plots Connections Help Viewer Presentation

Zoom Export Publish



Activities RStudio sön 15:36 sv

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins Project: (None)

Kmeans_from_scratch.R

Source on Save Run Source

```
12 par(mfrow=c(2, 2))
13 K <- 3
14 set.seed(123)
15 centers <- data[sample(1:dim(data)[1], K), ]
16 plot(data, xlab = "X", ylab = "Y", pch = 19)
17 points(centers, col="blue", cex = 3, pch = 19)
18 for(t in 1:3)
19 {
20 my_labels <- vector()
21 for(i in 1:dim(data)[1])
22 {
23   my_dist <- vector()
24   for(j in 1:K)
25   {
26     my_dist[j] <- sqrt((data[i, 1] - centers[j, 1])^2 + (data[i, 2] - centers[j, 2])^2)
27   }
28   my_labels[i] <- which.min(my_dist)
29 }
30 plot(data, xlab = "X", ylab = "Y", pch = 19)
31 points(data, col = my_labels, pch = 19)
32 points(centers, col = "blue", cex = 3, pch = 19)
33
34 new_centers <- list()
35 for(i in unique(my_labels))
36 {
37   new_centers[[i]] <- colMeans(data[my_labels == i,])
38 }
39 centers <- Reduce("rbind", new_centers)
40 }
```

33:1 (Top Level) R Script

Console Terminal Background Jobs

R 4.2.3 · ~/

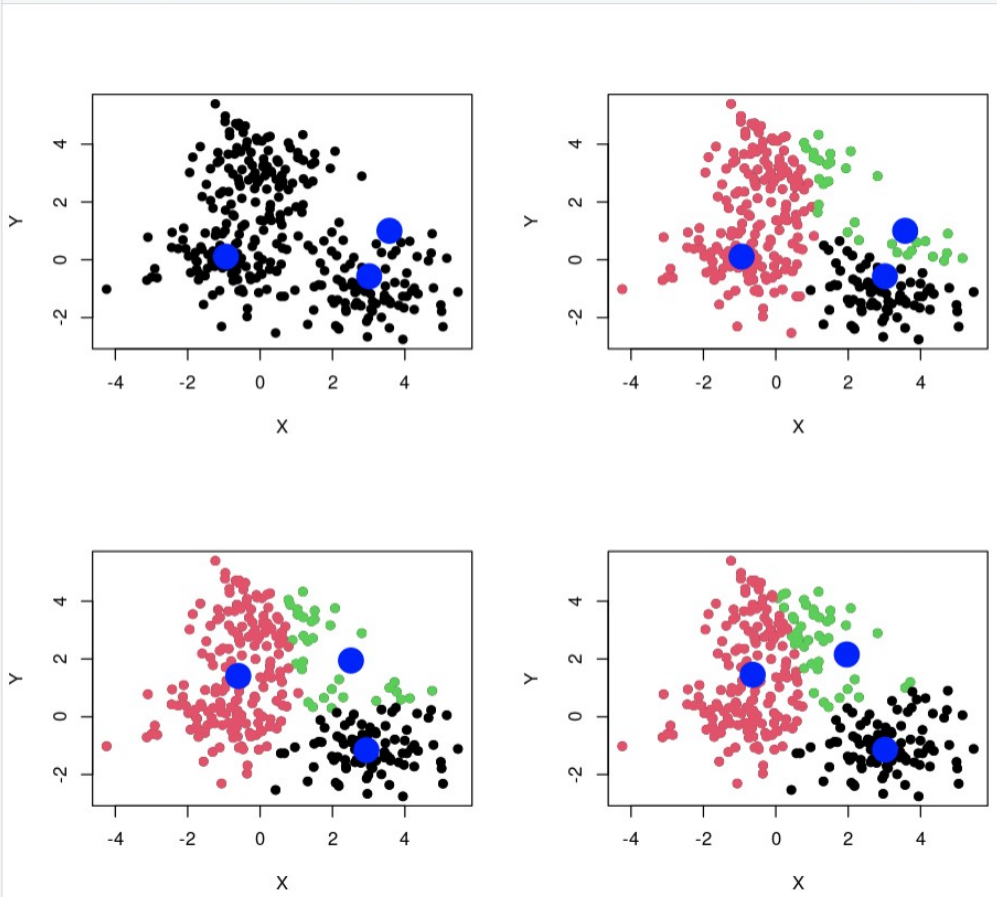
```
+
+ new_centers <- list()
+ for(i in unique(my_labels))
+ {
+   new_centers[[i]] <- colMeans(data[my_labels == i,])
+ }
+ centers <- Reduce("rbind", new_centers)
+ }
>
```

Environment History Files Packages

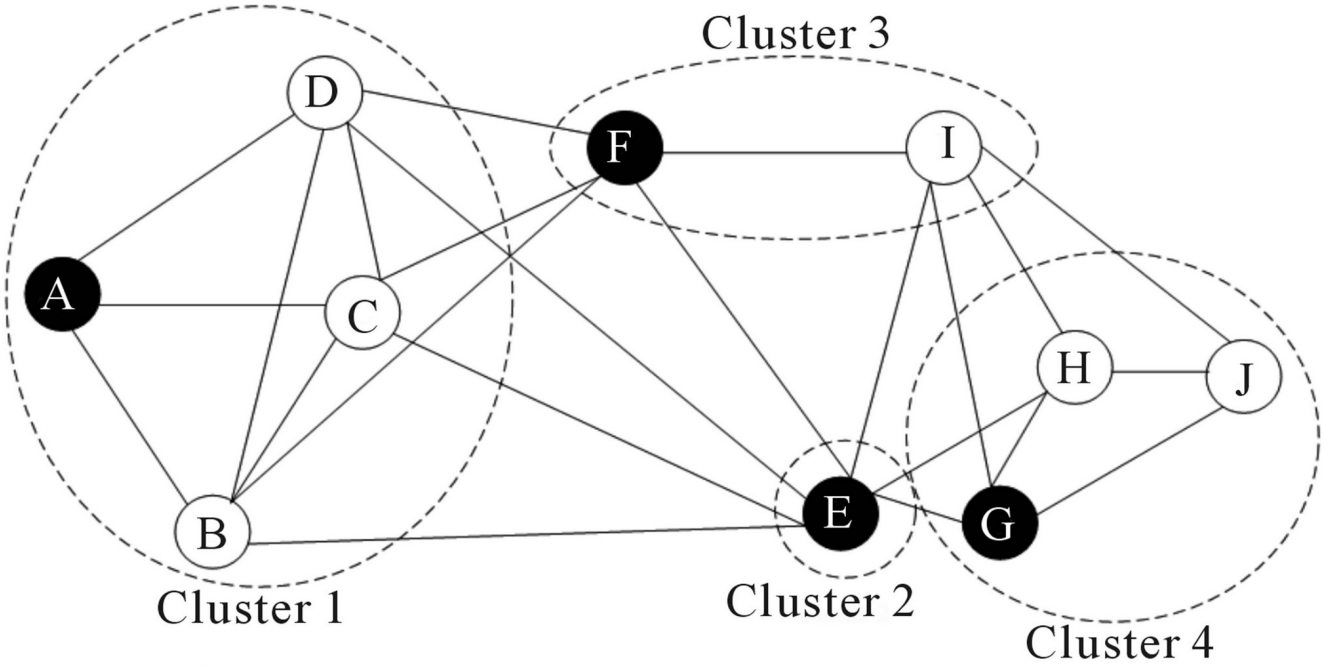
R Global Environment

Plots Connections Help Viewer Presentation

Zoom Export Publish



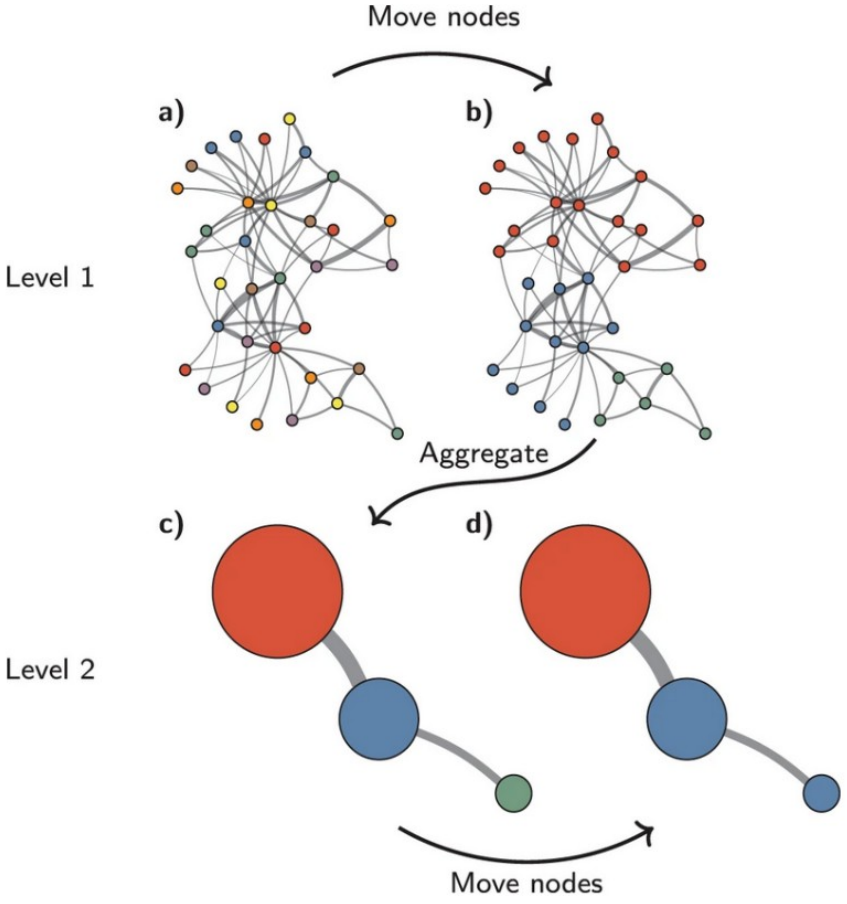
Shared Nearest Neighbors (SNN)



C and D belong to the same cluster because they share 4 neighbors (A, B, E and F)

C and F belong to different clusters because they share 3 neighbors (D, B and E)

Louvain clustering



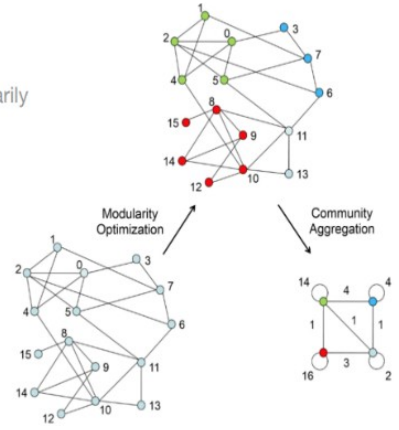
Louvain algorithm. The Louvain algorithm starts from a singleton partition in which each node is in its own community (**a**). The algorithm moves individual nodes from one community to another to find a partition (**b**). Based on this partition, an aggregate network is created (**c**). The algorithm then moves individual nodes in the aggregate network (**d**). These steps are repeated until the quality cannot be increased further.

Phase 1: greedy modularity optimisation

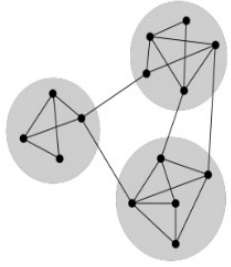
1. Start with 1n/community
2. Compute Q by moving i to the community of j
3. If ΔQ>1, node is placed in community
4. Repeat 1-3 until no improvement is found. Ties solved arbitrarily

Phase 2: coarse grained community aggregation

5. Link nodes in a community into single node.
6. Self loops show intra-community associations
7. Inter-community weights kept
8. Repeat phase 1 on new network



Other methods:
Walktrap
Label propagation
...
(benchmarking)



Campigotto 2014
Traag 2019

Source: https://nbisweden.github.io/workshop_omics_integration/session_topology

$$Q = \frac{1}{2m}\sum_c \left(e_c - \gamma\frac{K_c^2}{2m}\right)$$ - modularity optimization

Leiden clustering

scientific reports

Explore content | About the journal | Publish with us

nature > scientific reports > articles > article

Article | Open access | Published: 26 March 2019

From Louvain to Leiden: guaranteeing well-connected communities

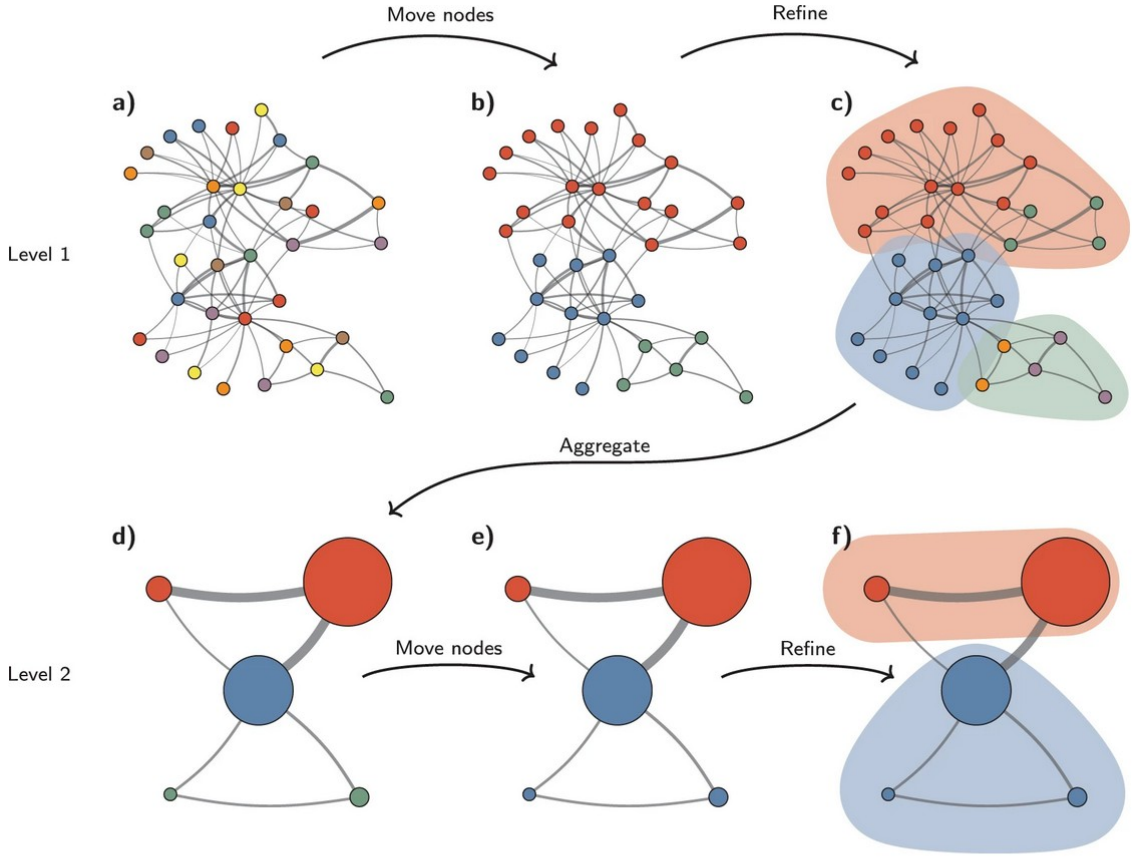
V. A. Traag, L. Waltman & N. J. van Eck

Scientific Reports **9**, Article number: 5233 (2019) | Cite this article

164k Accesses | **1907** Citations | **236** Altmetric | Metrics

Abstract

Community detection is often used to understand the structure of large and complex networks. One of the most popular algorithms for uncovering community structure is the so-called Louvain algorithm. We show that this algorithm has a major defect that largely went unnoticed until now: the Louvain algorithm may yield arbitrarily badly connected communities. In the worst case, communities may even be disconnected, especially when running the algorithm iteratively. In our experimental analysis, we observe that up to 25% of the communities are badly connected and up to 16% are disconnected. To address this problem, we introduce the Leiden algorithm. We prove that the Leiden algorithm yields communities that are guaranteed to be connected. In addition, we prove that, when the Leiden algorithm is applied iteratively, it converges to a partition in which all subsets of all communities are locally optimally assigned. Furthermore, by relying on a fast local move approach, the Leiden algorithm runs faster than the Louvain algorithm. We demonstrate the performance of the Leiden algorithm for several benchmark and real-world networks. We find that the Leiden algorithm is faster than the Louvain algorithm and uncovers better partitions, in addition to providing explicit guarantees.



Leiden algorithm. The Leiden algorithm starts from a singleton partition (**a**). The algorithm moves individual nodes from one community to another to find a partition (**b**), which is then refined (**c**). An aggregate network (**d**) is created based on the refined partition, using the non-refined partition to create an initial partition for the aggregate network. For example, the red community in (**b**) is refined into two subcommunities in (**c**), which after aggregation become two separate nodes in (**d**), both belonging to the same community. The algorithm then moves individual nodes in the aggregate network (**e**). In this case, refinement does not change the partition (**f**). These steps are repeated until no further improvements can be made.

scRNAseq Cancer-Associated Fibroblasts

nature communications

Explore content · About the journal · Publish with us

nature › nature communications › articles › article

Article | Open access | Published: 04 December 2018

Spatially and functionally distinct subclasses of breast cancer-associated fibroblasts revealed by single cell RNA sequencing

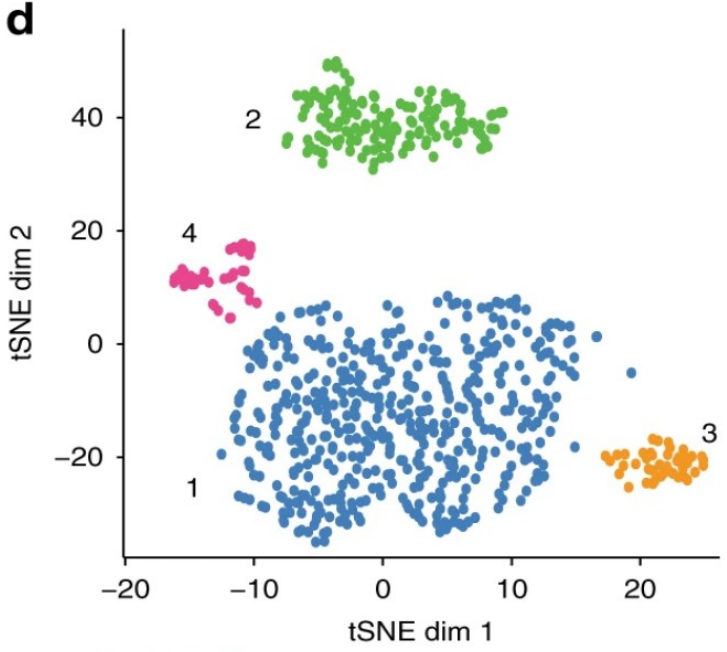
Michael Bartoschek, Nikolay Oskolkov, Matteo Bocci, John Lövrot, Christer Larsson, Mikael Sommar, Chris D. Madsen, David Lindgren, Gyula Pekar, Göran Karlsson, Markus Ringnér, Jonas Bergh, Åsa Björklund & Kristian Pietras

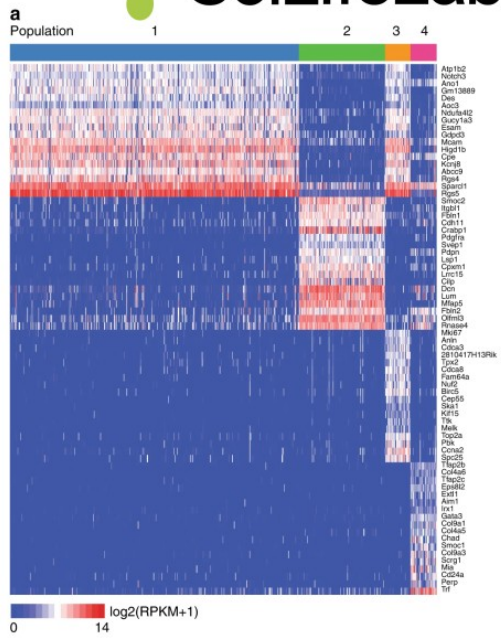
Nature Communications **9**, Article number: 5150 (2018) | Cite this article

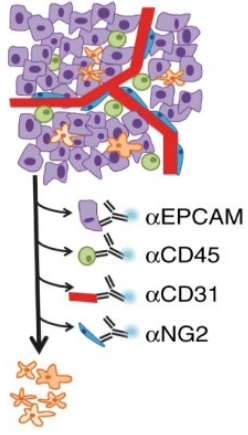
49k Accesses | **495** Citations | **91** Altmetric | Metrics

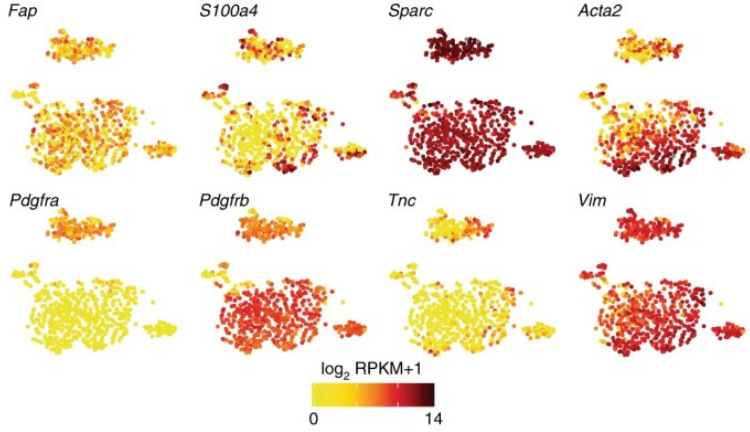
Abstract

Cancer-associated fibroblasts (CAFs) are a major constituent of the tumor microenvironment, although their origin and roles in shaping disease initiation, progression and treatment response remain unclear due to significant heterogeneity. Here, following a negative selection strategy combined with single-cell RNA sequencing of 768 transcriptomes of mesenchymal cells from a genetically engineered mouse model of breast cancer, we define three distinct subpopulations of CAFs. Validation at the transcriptional and protein level in several experimental models of cancer and human tumors reveal spatial separation of the CAF subclasses attributable to different origins, including the peri-vascular niche, the mammary fat pad and the transformed epithelium. Gene profiles for each CAF subtype correlate to distinctive functional programs and hold independent prognostic capability in

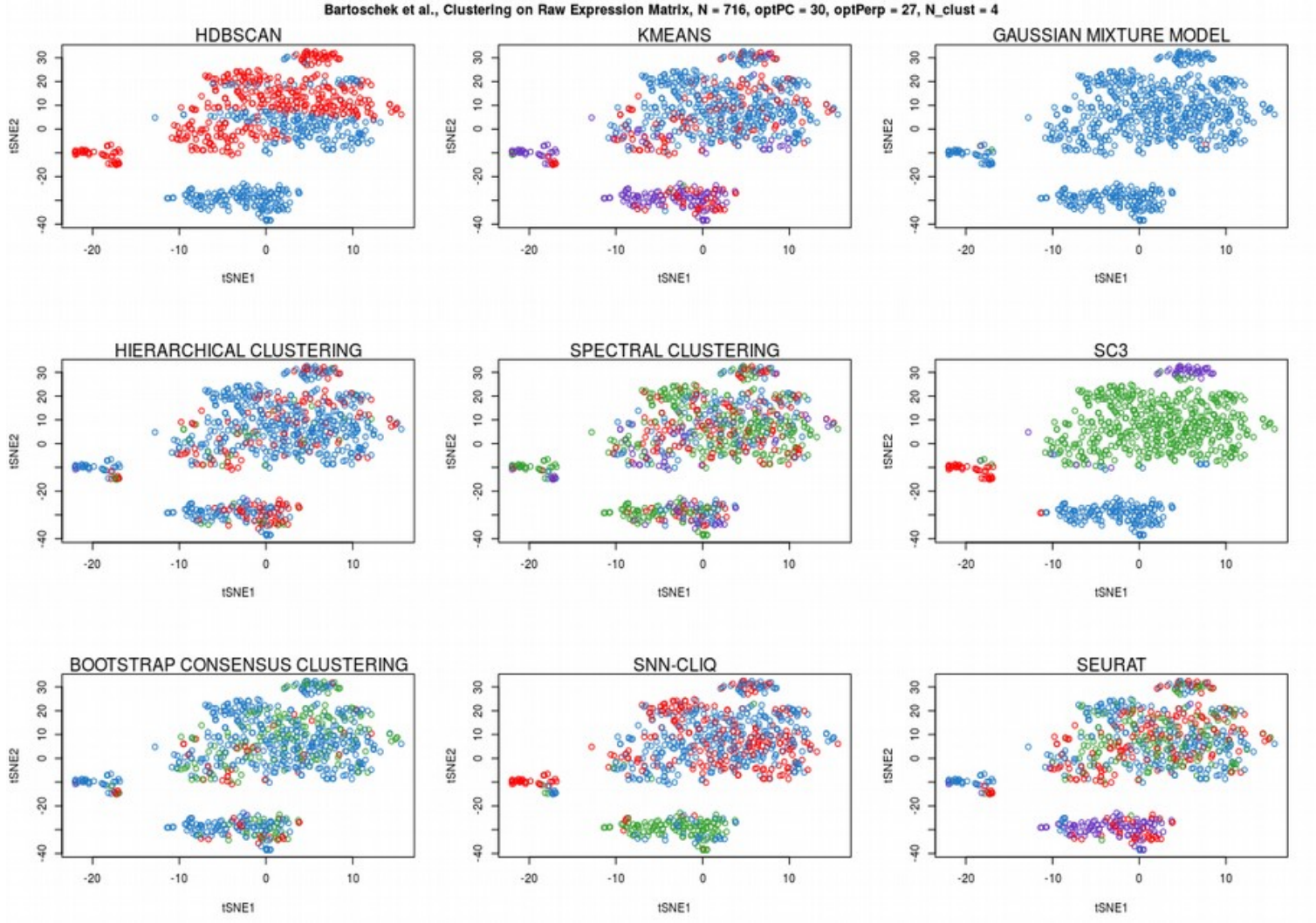




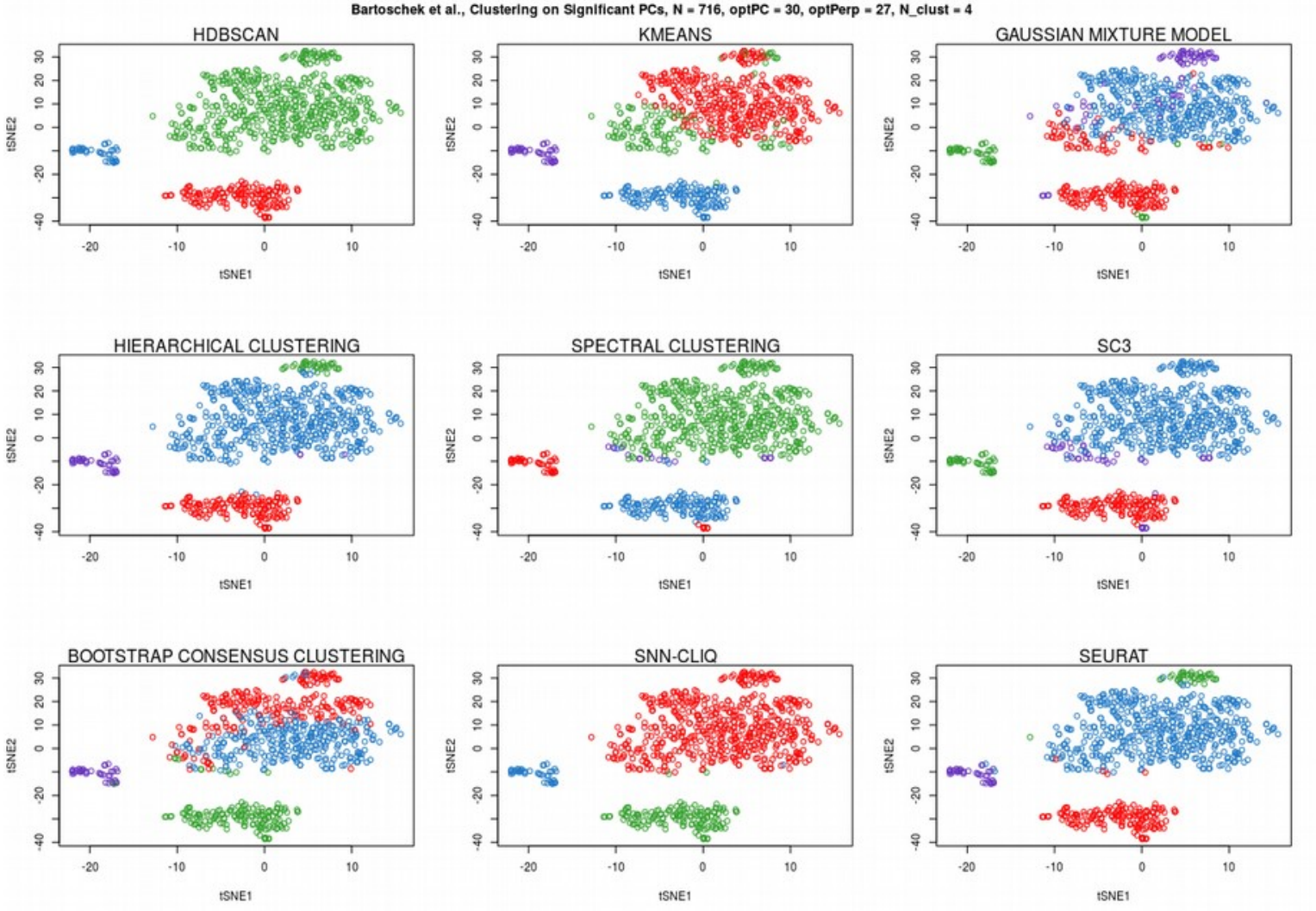




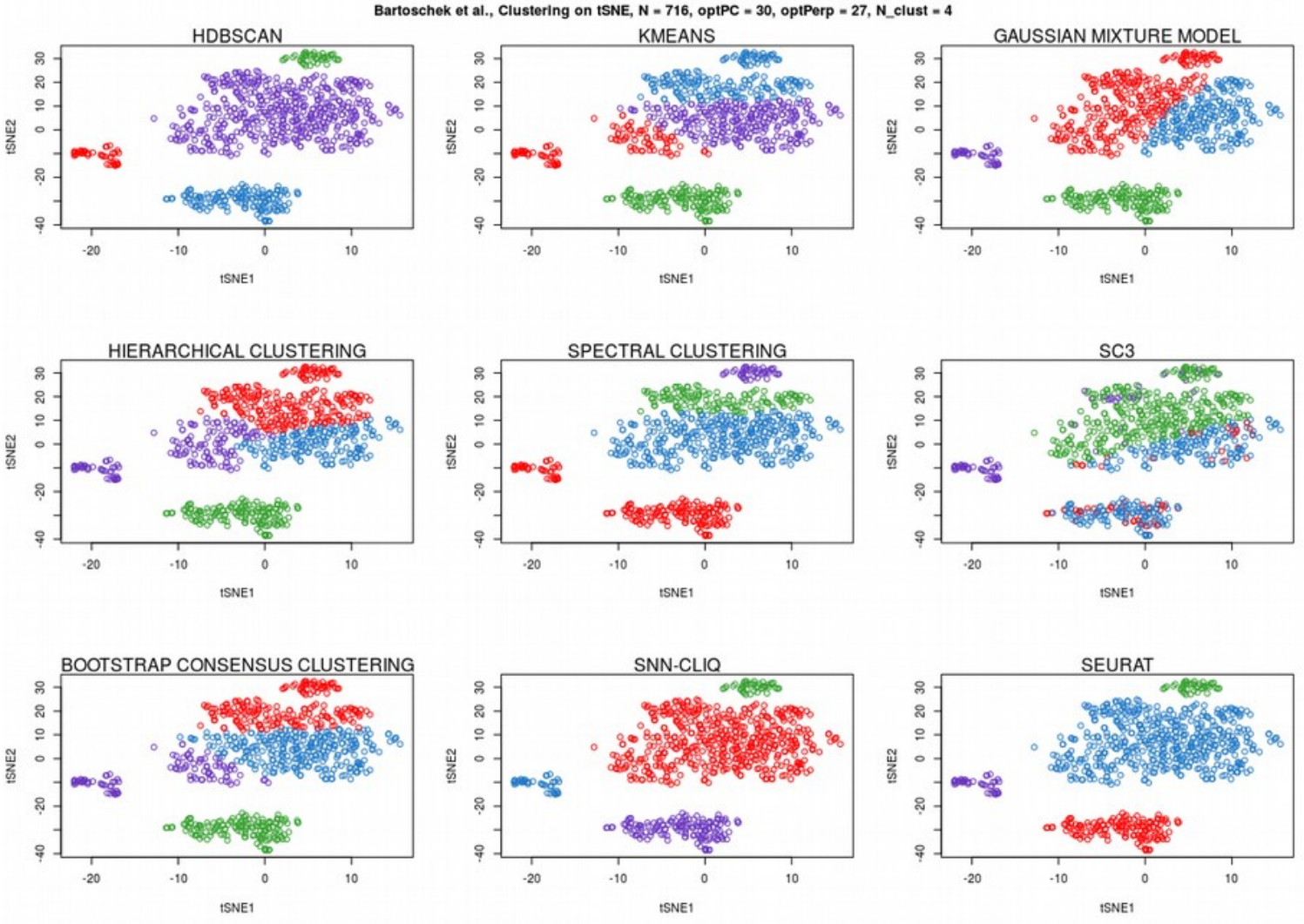
Clustering on Raw Expression Matrix



Clustering on leading PCA components



Clustering on two tSNE dimensions



Session summary

Take home messages of the session:

1) Clustering does not always agree with dimension reduction due to the curse of dimensionality

2) Hierarchical clustering is too sensitive to noise, and partitioning clustering is too sensitive to cluster geometry

3) Graph-based and density-based clustering are preferable for complex single cell data

National Bioinformatics Infrastructure Sweden (NBIS)



Knut och Alice Wallenbergs Stiftelse

Vetenskapsrådet

LUNDS UNIVERSITET

Acknowledgments: NIRI + TARGETWISE



European Regional Development Fund project “Functional Omics Analysis of Metabolic Diseases to Advance Drug Discovery Research Excellence in LIOS - TARGETWISE” (No.1.1.1.5./2/24/A/003)

2027 National Development Plan

Funded by the European Union

Acknowledgments: Metabolic Research Group



Open positions: fully covered PhD and 3-year postdoctoral fellow positions!